

Week 2 Day 4: Runtime Config, Observability, Failure Drill

Overview

Day 4는 Day 3에서 만든 image를 실제 실행 조건 위에 올려 보고, 정상/장애를 증거로 판단하는 날이다. 오늘은 Compose 본수업이 아니다. Compose는 Day 5에서 다룬다. 다만 마지막 8교시는 Prometheus/Grafana preview로 열어 두어 logs, exec, stats가 metrics/logs observability로 확장되는 그림을 미리 본다.

오늘의 핵심 질문은 다음과 같다.

container가 Up이면 정말 서비스가 정상인가? 아니라면 어떤 출력과 명령으로 원인을 좁힐 것인가?

Day 4 Boundary

포함	제외
docker run runtime option	Compose architecture 본실습
-e, --env-file, .env.example	compose.yaml 작성
logs, inspect, exec, stats	Kubernetes/Cloud observability 심화
missing env, wrong port, wrong network, stale volume, bad image tag	복잡한 MSA 장애
cleanup/data 삭제 판단	무조건 system prune
Day 5 Compose mapping preview, 8교시 Prometheus/Grafana 맛보기	Day 5 내용을 미리 끝내기

Learning Goals

- env/config를 image 밖에서 주입하고, 적용 증거를 남긴다.
- .env.example과 실제 .env를 구분하고 secret 값을 노출하지 않는다.
- docker logs로 startup/readiness/error를 구분한다.
- docker inspect로 port, env, network, mount, restart policy를 선별 확인한다.
- docker exec로 container 내부 process/filesystem/network 관찰을 수행한다.
- docker stats와 restart policy로 resource/crash 관찰 기준을 만든다.
- 8교시 preview로 Prometheus/Grafana/cAdvisor/Loki가 logs/metrics를 어떻게 보여주는지 맛본다.
- missing env, wrong env file path, wrong port, wrong network, stale volume, bad image tag를 실패 출력으로 분류한다.

- container/image/network/volume cleanup의 삭제 범위를 구분한다.
- Day 5에서 compose.yaml로 옮길 option mapping을 표로 남긴다.

Lesson Index

교시	주제	고유 판단 기준
1교시	runtime config, env file, secret masking	env가 image가 아니라 실행 시점에 들어가고 안전하게 기록되는가
2교시	logs와 HTTP 정상 확인	Up, log, HTTP 응답을 분리해 보는가
3교시	inspect/exec 내부 확인	Docker metadata와 container 내부 상태를 구분하는가
4교시	stats/restart/crash loop	resource 관찰과 restart policy의 한계를 설명하는가
5교시	failure drill	실패 출력에서 첫 확인 명령을 고르는가
6교시	cleanup/security audit	지울 자원과 보존할 data를 구분하는가
7교시	Compose mapping handoff	긴 run option을 compose 항목으로 옮길 준비가 됐는가
8교시	Prometheus/Grafana observability preview	logs와 metrics의 차이를 설명하는가

Practice Files And Assets

자료	용도
assets/day4-runtime-observability-overview.png	Day 4 runtime/observability 전체 구조
assets/lesson-02-env-secret-masking.png	.env.example, .env, secret masking 시각화
assets/lesson-03-logs-http-verification.png	logs와 HTTP 200 확인 지점
assets/lesson-04-inspect-exec-evidence.png	inspect와 exec 확인 위치 비교
assets/lesson-05-stats-restart-crash-loop.png	stats, restart policy, crash loop 구분
assets/lesson-06-runtime-failure-rca.png	runtime failure drill과 RCA
assets/lesson-07-cleanup-security-audit.png	cleanup/security audit와 volume 삭제 위험
assets/lesson-08-runtime-to-platform-mapping.png	Docker run에서 Compose/Kubernetes/Terraform으로 이어지는 mapping
labs/env-report/report.sh	env 주입 결과를 출력하는 작은 실습 스크립트
labs/env-report/.env.example	공유 가능한 env file 예시
labs/observability-preview/README.md	Prometheus, Grafana, cAdvisor, Loki preview
labs/observability-preview/compose.yaml	metrics/logs preview stack
hands-on-lab.md	Day 4 전체 실행 흐름
Day 3 image 또는 nginx:1.27-alpine	runtime/logs/inspect/exec 실습 대상
postgres:16-alpine	missing env, volume, network 실패 드릴 대상

labs/compose-app은 Day 5 Compose preview 성격의 자료다. labs/observability-preview는 8교시 logs/metrics preview 자료다. Day 4 본 실습의 앞부분은 여전히 docker run 기반으로 진행한다.

Common Rules

- docker ps의 Up은 process 상태다. 서비스 정상 여부는 log, HTTP, DB query 같은 별도 확인이 필요하다.
- secret 값은 README, screenshot, terminal output에 남기지 않는다. 문서에는 key 이름과 masking 된 예시만 남긴다.
- logs, inspect, exec는 관찰 도구지만 secret 유출 경로가 될 수도 있다. 출력 전체를 붙이지 말고 필요한 줄만 masking해서 남긴다.

- 환경설정 파일은 `docker run --env-file ./path/to/.env ...`로 로드한다. 파일은 `KEY=value` 형식을 기본으로 하고, `#`로 시작하는 줄은 comment로 본다.
- `-e KEY=value`는 한두 개 값을 빠르게 실험할 때, `--env-file`은 여러 환경설정을 파일로 묶어 반복 실행할 때 사용한다.
- `.env`는 실행용 local 파일이고, `.env.example`은 공유용 형식 문서다. 실제 secret 값은 `.env.example`에 넣지 않는다.
- `.env.dev`, `.env.staging`, `.env.prod`처럼 환경별 파일을 나눌 수 있다. 이 원칙은 이후 Kubernetes ConfigMap/Secret, Terraform `*.tfvars` 환경 분리로 이어진다.
- 실제 사고에서도 GitHub 자격증명 노출, cloud access key 관리 부실, secret 하드코딩은 DB 접근 사고로 이어질 수 있는 핵심 쟁점이 된다. 공개 repository와 image layer에는 실제 credential을 남기지 않는다.
- env file을 수정해도 이미 만들어진 container 환경이 자동으로 바뀌지 않는다. 새 값을 적용하려면 container를 다시 생성한다.
- `inspect`는 전체 JSON dump를 복사하지 않는다. 문제와 관련된 field만 뽑아 본다.
- `exec`는 container 내부 관찰 도구다. host와 container 내부 command availability가 다를 수 있다.
- `docker exec -it <container> sh` shell 진입은 상태 확인 목적일 때만 허용한다. container 내부 파일 수정, 삭제, package 설치, 임시 hotfix는 금지한다.
- container 내부에서 수정해야 할 것처럼 보이면 shell에서 고치지 말고 Dockerfile, env file, bind mount 원본, run option, compose.yaml 같은 재현 가능한 source로 돌아간다.
- failure drill 뒤에는 실패 container, 임시 network, 실습 volume이 다음 실습을 방해하지 않게 cleanup한다.
- named volume 삭제는 data 삭제다. `volume rm, down -v, system prune --volumes`는 기본 cleanup이 아니다.

Instructor Rule

Day 4는 명령어 암기 시간이 아니다. 각 실습은 증상 -> 첫 확인 명령 -> 확인한 출력 -> 원인 범주 -> 수정/보존 판단 순서로 말하게 한다. 특히 secret, env, volume, cleanup은 출력 전체를 붙이는 방식이 아니라 필요한 key와 masked value만 남기게 한다.

End-Of-Day Checklist

- [] `-e`와 `--env-file` 중 하나 이상으로 runtime config를 주입했다.
- [] `.env.example`과 실제 `.env` 차이를 설명했다.
- [] secret 값을 출력물에 남기지 않았다.
- [] logs로 정상/장애 신호를 확인했다.
- [] `inspect`로 port/env/network/mount/restart 중 2개 이상 확인했다.
- [] `exec`로 내부 process 또는 filesystem을 확인했다.
- [] `exec shell`에서 read-only 확인과 금지 행동을 구분했다.
- [] stats 또는 restart policy로 resource/crash 관찰을 했다.
- [] failure drill 3개 이상을 출력과 함께 RCA로 정리했다.

- [] cleanup audit에서 container/image/network/volume을 구분했다.
- [] Day 5 Compose mapping 표를 작성했다.
- [] Prometheus/Grafana preview에서 docker stats와 metrics timeline의 차이를 설명했다.

Day 5 Connection

Day 5는 Compose 확정일이다. Day 4에서 긴 docker run option을 충분히 겪은 뒤, 다음 mapping을 Day 5에서 compose.yaml로 옮긴다.

Day 4 docker run	Day 5 Compose
-p 18084:80	services.web.ports
-e KEY=value	services.*.environment
--env-file .env	env_file 또는 variable interpolation
-v volume:/path	services.*.volumes
--network app-net	networks
docker logs	docker compose logs
docker exec	docker compose exec

Completion Definition

Day 4 완료는 다음 조건을 만족할 때 선언한다.

1. runtime command와 적용된 config를 설명할 수 있다.
2. logs/inspect/exec/stats 중 상황에 맞는 관찰 명령을 고를 수 있다.
3. 실패 출력에서 env/port/network/volume/image 중 원인 범주를 좁힐 수 있다.
4. cleanup이 data 삭제인지 아닌지 구분할 수 있다.
5. Day 5 compose.yaml로 옮길 option mapping이 있다.